

OpenRouter Large Model API Aggregation Platform - Text Interaction

OpenRouter Large Model API Aggregation Platform - Text Interaction

Concept Introduction

1.1 What is "OpenRouter"?

2. Practical Operations

2.1 Obtaining API Credentials

2.2 Step-by-Step Guide with Images

2.3 Configuration File Modification

Concept Introduction

1.1 What is "OpenRouter"?

OpenRouter is a **Large Model API Aggregation Platform**. You can think of it as a "universal remote control for large language models". Normally, if you want to use OpenAI's GPT-4, Anthropic's Claude-3, and Google's Gemini, you would need to register accounts on three separate platforms, obtain three sets of API keys, and write three different sets of API call code tailored to each platform's unique format.

OpenRouter solves this problem. It provides a **unified API interface** (compatible with OpenAI's API format). With just one OpenRouter API key, you can access all the models mentioned above plus hundreds of other open-source and closed-source models through a single entry point.

The **core advantages** of integrating OpenRouter into the `largemodell` project are:

- **Ultimate Flexibility:** You can seamlessly switch between GPT, Claude, Llama, and any other model by simply changing the model name in the configuration file, without modifying any code.
- **Cost Efficiency:** OpenRouter provides clear pricing for each model, making it easy to compare and select the most cost-effective model for specific tasks.
- **Explore Cutting-Edge Models:** Many new and excellent open-source models are added to OpenRouter first, allowing developers to quickly try out new releases.

In the `largemodell` project, you can enable this powerful aggregation platform by configuring the `llm_platform` parameter to `openrouter`.

2. Practical Operations

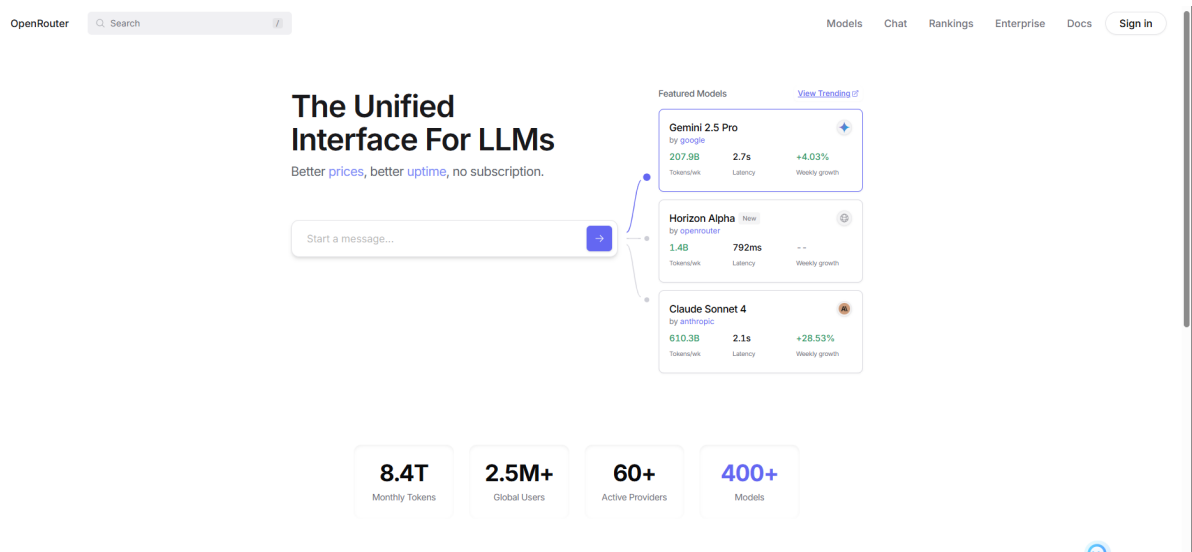
2.1 Obtaining API Credentials

1. **Register for OpenRouter:** Visit [OpenRouter.ai](https://openrouter.ai) and register an account using Google or email. If the link is broken, you need to search for the "OpenRouter" platform yourself.
2. **Add Credits:** OpenRouter uses a prepaid model. You need to add a small amount of credits (e.g., \$5) on the "Credits" page before you can start using it. Alternatively, you can use their free models directly, but there are rate limits.
3. **Create an API Key:** On the "Keys" page, click "Create Key" to generate a new API key, and **immediately copy and save it**.

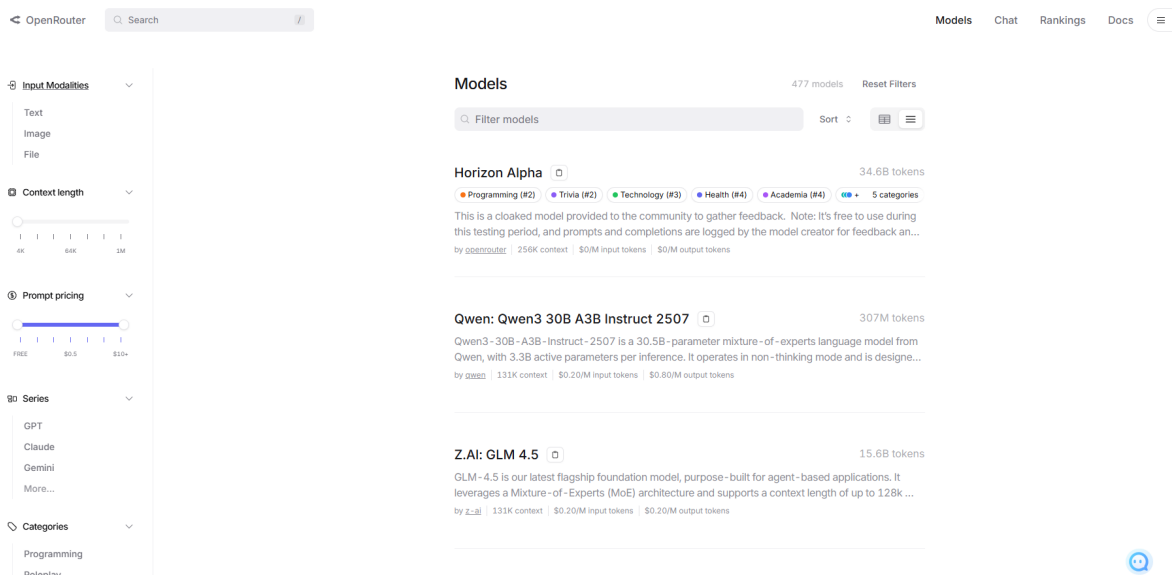
2.2 Step-by-Step Guide with Images

1. First, visit the following website. If the link is broken, you need to search for the "OpenRouter" platform yourself and then look for keywords related to large language models.

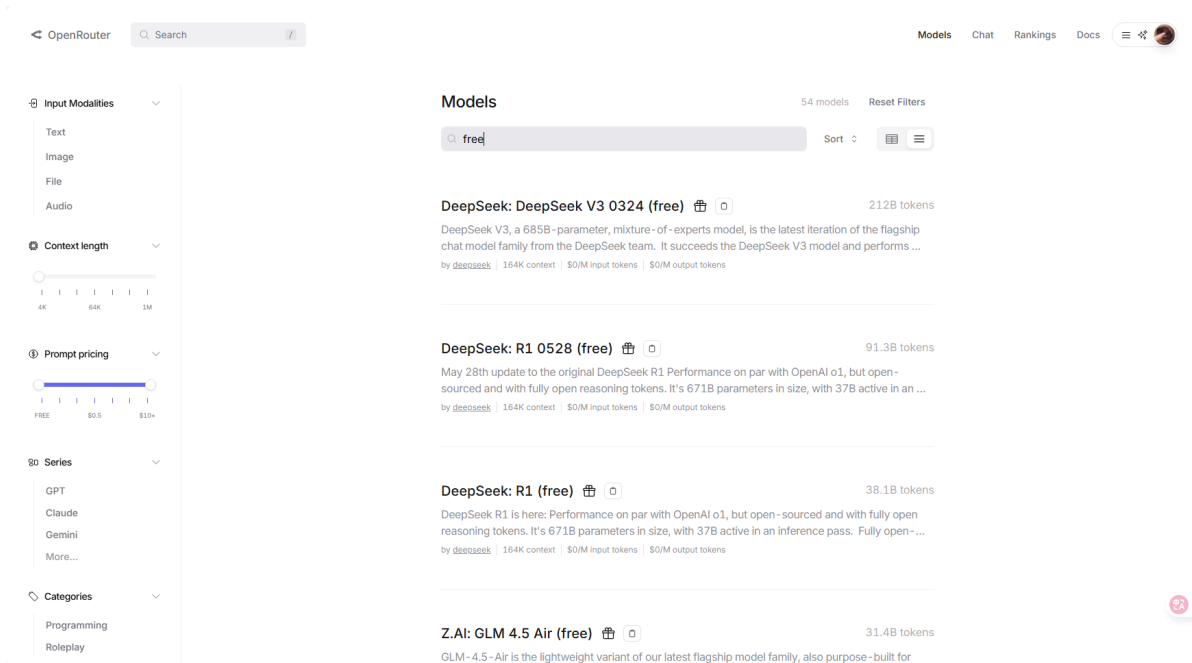
[OpenRouter](https://openrouter.ai)



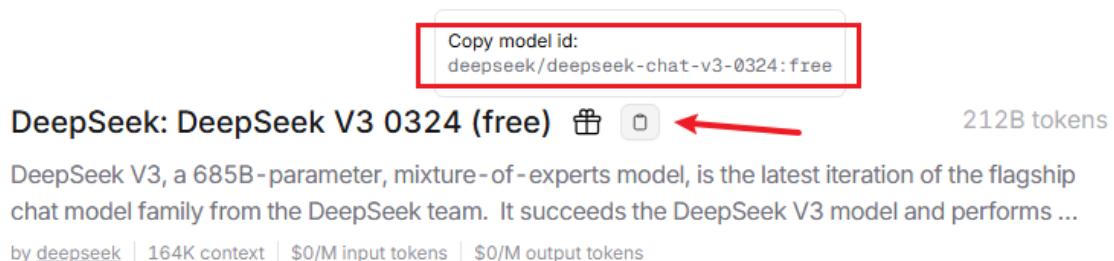
2. Register and log in to your account
3. After registering, click to enter the Model Plaza



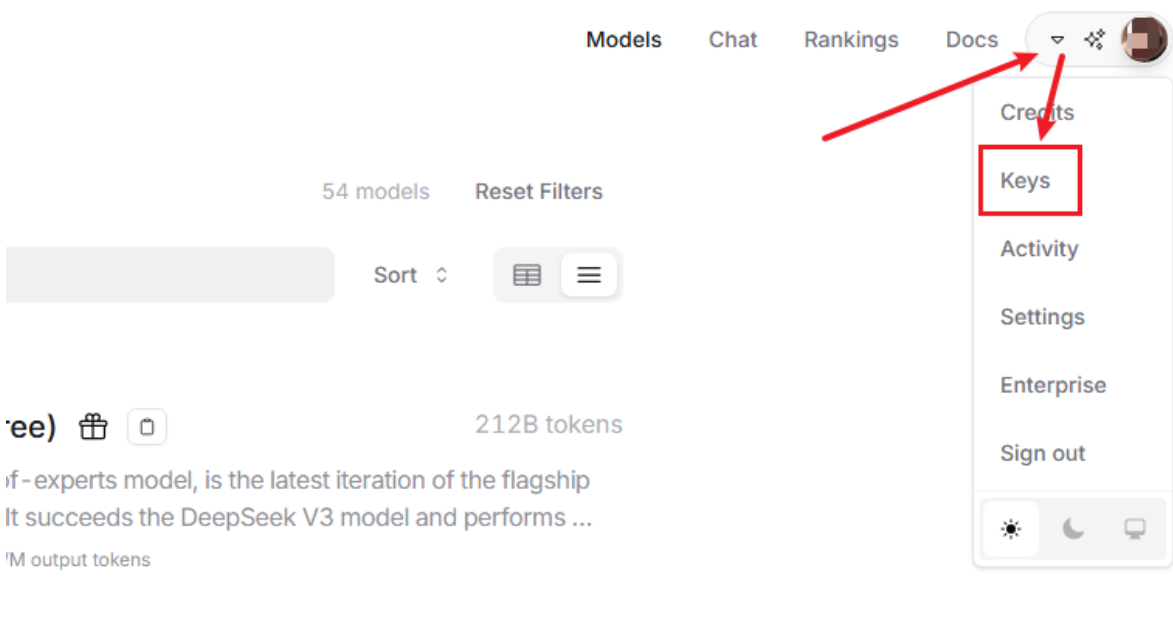
4. You can type "free" in the search bar to find models that can be called for free.



5. Hover your mouse over the clipboard icon next to a model, and you can see the name to use when calling the model. Click to copy it.



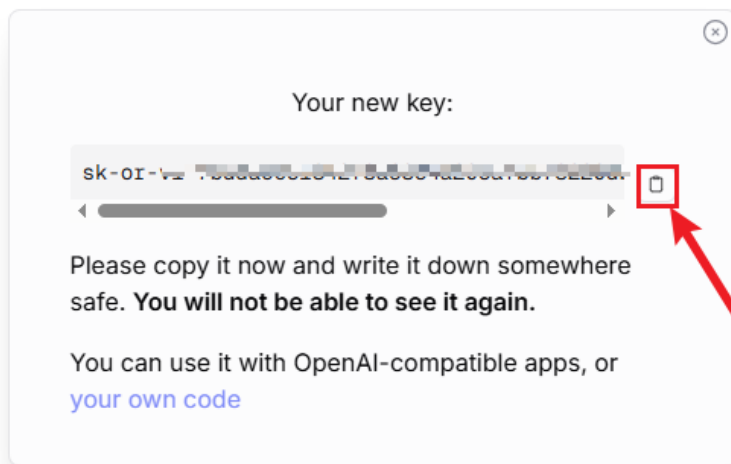
6. Move your mouse to the avatar in the upper right corner, and a menu will expand. You can click on "Keys" to create a key.



7. Click to apply for an API key

Manage your API keys to access all models from OpenRouter

8. After applying, you will receive an API key string



2.3 Configuration File Modification

1. **Open** `large_model_interface.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/large_model_interface.yaml
```

Then replace `sk-xxxxxxxxxx` in `openrouter_api_key` with the API key you just copied.

`openrouter_model` is used to configure the large model for conversations.

```
# OpenRouter Platform Configuration
openrouter_api_key: "sk-xxxxxxxxxxxxxxxxxxxx"
openrouter_model: "moonshotai/kimi-k2:free" # Model to use, e.g.,
"google/gemini-pro-vision"
```

2. Open `yahboom.yaml`:

```
vim ~/.yahboomcar_ws/src/largemodel/config/yahboom.yaml
```

3. **Switch Platform:** Change the `llm_platform` parameter to `openrouter`.

```
llm_platform: 'openrouter'
```

After switching the platform parameter, go back to the workspace and recompile for the changes to take effect.